

# Research Project Proposal

## Comparative Analysis of Numerical Interpolation Methods for Robot Trajectory Planning

### Introduction

Modern robotic systems—from industrial manipulators on factory floors to autonomous mobile robots navigating warehouses—share a fundamental computational challenge: given a sequence of desired positions (waypoints), how should the robot interpolate between them to produce motion that is physically feasible, smooth, and efficient? This question sits at the intersection of numerical methods and robotics engineering, and the answer has direct consequences for mechanical wear, energy consumption, payload stability, and whether the planned trajectory can be executed in real time.

In practice, a higher-level planner (such as a path planner or a human operator teaching waypoints) produces a sparse set of positions the robot must reach. The trajectory generation layer must then fill in the motion between those points. The simplest approach—connecting waypoints with straight-line segments—creates abrupt velocity changes at each waypoint that would require infinite acceleration, making it physically impossible to follow exactly. More sophisticated interpolation methods from numerical analysis address this by producing curves with guaranteed continuity in velocity, acceleration, or even jerk (the derivative of acceleration).

This project investigates how different numerical interpolation methods perform when applied to the specific problem of robot trajectory generation, using evaluation criteria drawn directly from what matters in real robotic systems.

### Problem Statement

A robot must traverse a series of waypoints while maintaining smooth, continuous motion. The choice of interpolation method determines the quality of the resulting trajectory across multiple dimensions: positional accuracy, velocity continuity, acceleration smoothness, jerk minimization, computational cost, and total path length. Selecting an inappropriate method can result in mechanical vibration, excessive motor torque demands, payload damage, or trajectories that cannot be computed fast enough for real-time control.

The core question this project addresses is: among the numerical interpolation methods covered in Chapters 18 and 24 of Chapra and Canale, which method (or combination of methods) produces the most suitable trajectories for robotic applications, and under what conditions does each method excel or fail?

## Proposed Solution

I propose a comparative computational study of four interpolation methods applied to robot trajectory generation:

- **Linear Interpolation:** Connects waypoints with straight-line segments. Serves as the baseline. Produces C continuity only (position is continuous but velocity is not).
- **Cubic Spline Interpolation:** Fits piecewise third-degree polynomials between waypoints with constraints ensuring continuity of position, first derivative (velocity), and second derivative (acceleration). This is the most widely used method in industrial robotics today.
- **Quintic Spline Interpolation:** Extends the cubic approach to fifth-degree polynomials, adding continuity of the third and fourth derivatives. This ensures smooth jerk profiles, which is important for vibration-sensitive applications and high-speed motion.
- **B-Spline Approximation:** A fundamentally different approach that does not force the curve to pass exactly through each waypoint. Instead, waypoints serve as control points that influence the curve shape. This provides local control (moving one waypoint only affects nearby segments) and inherent smoothness, at the cost of not hitting waypoints exactly.

In addition to comparing interpolation methods, I will use numerical differentiation techniques (forward difference, centered difference, and higher-order formulas from Chapter 28) to compute velocity, acceleration, and jerk profiles from the interpolated position data. This allows direct quantitative comparison of motion smoothness across methods.

## Research Methodology

1. Define two test scenarios representing realistic robotic tasks: (a) a 2D mobile robot

navigating around obstacles via a set of floor-plane waypoints, and (b) a 2-DOF planar robot arm executing a pick-and-place operation defined by joint-angle waypoints.

2. Implement each interpolation method in Python using NumPy and SciPy for matrix operations, with the core spline algorithms coded from the textbook formulations (not simply calling library black-box functions) to demonstrate understanding of the numerical procedures.

3. Generate interpolated trajectories at fine time resolution (1000+ points per trajectory) for each method and each test scenario.

4. Apply numerical differentiation (forward difference and centered difference formulas) to the position data to extract velocity, acceleration, and jerk profiles for each method.

5. Evaluate each method quantitatively using the following metrics: maximum velocity discontinuity, maximum acceleration, RMS jerk, total path length, and computation time.

6. Vary experimental conditions to test robustness: sparse waypoints (5 points) vs. dense waypoints (20 points), waypoints with sharp turns vs. gradual curves, and uniform vs. non-uniform spacing.

7. Produce comparative visualizations: overlaid trajectory plots, velocity/acceleration/jerk profiles, and summary tables of quantitative metrics.

8. Analyze results to draw conclusions about which method is best suited for which robotic application and under what conditions.

## Datasets

The waypoint datasets will be synthetically generated to represent realistic robotic scenarios:

- **Mobile robot scenario:** 8–15 waypoints representing navigation through a cluttered environment (e.g., warehouse aisle with turns). Coordinates generated to include a mix of straight segments, 90-degree turns, and gradual curves.
- **Robot arm scenario:** 6–10 joint-angle waypoints representing a pick-and-place cycle (home → approach → grasp → lift → move → place → retract → home). Values chosen within realistic joint limits for a 2-DOF planar arm.

Using synthetic data is deliberate: it allows controlled comparison by ensuring all methods are tested on identical inputs, and it allows systematic variation of difficulty (spacing,

curvature) without confounding factors.

## References

1. Chapra, S. C., & Canale, R. P. (2021). *\*Numerical Methods for Engineers\** (8th ed.). McGraw-Hill.
2. Craig, J. J. (2018). *\*Introduction to Robotics: Mechanics and Control\** (4th ed.). Pearson.
3. Biagiotti, L., & Melchiorri, C. (2008). *\*Trajectory Planning for Automatic Machines and Robots\**. Springer.
4. Siciliano, B., et al. (2009). *\*Robotics: Modelling, Planning and Control\**. Springer.